ARReSVG: Intelligent Multi-AAV Navigation in Partially Observable Spaces Using Adaptive Deep Reinforcement Learning Approach

Selvam Essaky, *Graduate Student Member, IEEE*, Gunasekaran Raja, *Senior Member, IEEE*, Kapal Dev, *Senior Member, IEEE*, and Dusit Niyato, *Fellow, IEEE*

***Abstract*—Autonomous Aerial Vehicles (AAVs) have brought about a revolution in various applications worldwide. Applications such as aerial surveillance, search and rescue, and operations beyond the line of sight rely on the autonomous navigation capabilities of multi-AAVs. This paper introduces a novel framework, Adaptive Rapid Recurrent Stochastic Valued Gradient (ARReSVG), empowered by Deep Reinforcement Learning (DRL) to address navigational challenges faced by AAVs in partially observable spaces. The proposed ARReSVG framework incorporates an adaptive learning approach to address issues related to partial observability-based autonomous navigation effectively. Compared with traditional obstacle avoidance techniques, the proposed navigation framework utilizes a distributed Multi-AAV Consensus Path Planning (MUCP2) mechanism to perform collaborative path planning for multiple AAVs to ensure collision avoidance trajectories. Additionally, the learned value-action policy implemented in the AAV alleviates the computational demands of real-time map-building. The ARReSVG framework leverages a Gated Recurrent Unit (GRU) to reduce computing complexity and accurately approximate the action-value function. The simulation results show that the ARReSVG framework achieves an impressive 94% success rate, enabling AAVs to navigate independently in unknown and complex environments.**

***Index Terms*—Multi-AAV navigation, consensus path planning, deep reinforcement learning (drl), stochastic value gradient, collaborative decision making.**

I. Introduction

IN LIGHT of tremendous advances in Artificial Intelligence (AI), Autonomous Aerial Vehicles (AAVs) have gained considerable attention as technologically advanced vehicles. Equipping AAVs with autonomous capabilities is essential for several Internet of Drones (IoD) applications, such as Smart Cities, Urban Planning, Search and Rescue operations [1]. This work aims to develop a technique that enables multi-AAV to move independently in an unfamiliar large-scale environment, including partially visible scenarios, from any starting position to any target point by avoiding collision efficiently. Numerous approaches, including learning and non-learning, have been proposed for AAV navigation [2]. Simultaneous Localization and Mapping (SLAM) [3] is a popular non-learning algorithm that assists AAVs in simultaneously mapping their environment and locating themselves inside it. The endurance of AAVs in facilitating sustained autonomy plays a pivotal role in navigational operations. However, SLAM algorithms are typically not well-suited to complex environments. It is crucial to thoroughly explore various ongoing challenges and possible solutions to SLAM problems [3].

Another category of non-learning-based strategies is sensing and avoidance. AAVs must implement collision-free path planning facilitated by an artificial potential field method to navigate safely through dynamic environments by avoiding obstacles [4]. Most sense and avoidance strategies are highly challenging for AAVs in large-scale and complex environments. Recently, vision-based approaches have played a significant role in autonomous navigation, particularly in sense-and-avoid operations. Many target search applications use forward-looking camera-enabled autonomous AAV navigation for real-time obstacle tracking [5]. This capability enhances the autonomy of AAVs to a certain extent. In recent years, the commercial influence of micro-AAVs has demonstrated impressive results in target-tracking and path-planning applications [6].

Consequently, the most common limitation of navigation operations is collision avoidance. Obstacle avoidance becomes more crucial as the environment increases. Hence, an active collision-free 3D deployment of multi-AAV is highly essential [7]. Advancements in variants of Reinforcement Learning (RL) algorithms [8], particularly Deep Reinforcement Learning (DRL), have influenced AAV autonomy. Generally, DRL algorithms take advantage of DL and RL algorithms. This integration significantly improves the learning of advantageous behaviors for agent training while also leading to swarm intelligence [9].

Received 7 October 2024; revised 14 February 2025 and 16 May 2025; accepted 22 May 2025. Date of publication 26 May 2025; date of current version 17 October 2025. This work was partly supported by the Erasmus+ KA107 Programme (ICM) and partly by NGNLab, Department of Computer Technology, Anna University, MIT Campus, Chennai - 600044, India. The review of this article was coordinated by Prof. Jun Zheng. *(Corresponding author: Gunasekaran Raja.)*

Selvam Essaky and Gunasekaran Raja are with the NGNLab, Department of Computer Technology, Anna University, MIT Campus, Chennai 600044, India (e-mail: selvame@ieee.org; dr.r.gunasekaran@ieee.org).

Kapal Dev is with the Department of Computer Science, Munster Technological University, T12 P928 Bishopstown, Cork, Ireland, also with the Department of insbtute of intelligent systems, University of Johannesburg, Auckland Park 2006, South Africa, and also with the Department of Electrical and Computer Engineering, Lebanese American University, Byblos 1111, Lebanon (e-mail: kapal.dev@ieee.org).

Dusit Niyato is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: dniyato@ntu.edu.sg).

Digital Object Identifier 10.1109/TVT.2025.3573898

0018-9545 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

However, AAVs encounter intricate challenges in scenarios with partial observability, where they autonomously navigate expansive areas marked by dynamic terrain, obstacles, and unforeseen circumstances. The multi-AAV confrontation issues [10] can be effectively addressed by the graph attention-based multi-agent actor-critic reinforcement learning approach, particularly when performing target-prediction tasks. DRL algorithms do a paradigm shift by empowering AAVs to acquire intricate navigation strategies directly from sensory and Inertial Measurement Unit (IMU) inputs. Beyond merely enhancing adaptability and efficiency in AAV navigation, DRL algorithms foster continual learning, allowing multi-AAV to refine their navigation skills over time. Incorporating the proposed Adaptive learning-based Rapid Recurrent Stochastic Valued Gradient (ARReSVG) algorithm into multiple AAV navigation signifies a momentous advancement, offering unprecedented autonomy.

The main contributions of this paper are summarized as follows.

1) An Adaptive learning-based Rapid Recurrent Stochastic Valued Gradient (ARReSVG) framework is proposed to enable autonomous multi-AAV navigation in scenarios with partially observable spaces. It integrates an adaptive learning mechanism to mitigate uncertainties stemming from partial observability constraints, aiding AAVs in reaching the target state safely.
2) A Gated Recurrent Unit (GRU)-based Actor-Critic network is introduced for approximating the action-value function using sequential observations. Furthermore, the GRU Net enhances the cumulative rewards across diverse environments over time.
3) A quantitative evaluation of GRU Net has been conducted to analyze temporal dependencies among observations and effectively approximate values. Furthermore, the experimental results suggest that the GRU utilized in this work surpasses other baseline DRL algorithms related to computational throughput.

The remainder of this paper is organized as follows. Section II presents a literature review of multi-AAV autonomous navigation and the existing DRL algorithms. Section III outlines the system model, which highlights the problem formulation and partial observability scenario for multi-AAV navigation. Section IV addresses the proposed methodology, encompassing the ARReSVG framework, GRU Network unit, and consensus path-planning mechanism for multi-AAV. Section V presents the analysis and discussion of the results. Finally, Section VI concludes the paper.

II. Related Work

This section reviews the existing studies on autonomous AAV navigation with a focus on partial observability [11]. AAV trajectory planning and navigation have been extensively studied, and there is an increasing interest in multi-AAV systems for search, localization, and path planning. AAV search and localization techniques play a vital role in identifying unknown airborne and ground objects in both visual and non-visual line-of-sight aspects [12]. Traditional non-learning approaches often fail in complex and dynamic environments. Because AAV navigation problems have been recognized as sequential decision-making tasks, Reinforcement Learning (RL) algorithms have gained significant attention [9], [13].

Deep Reinforcement Learning (DRL) has resulted in considerable advancements in AAV trajectory planning and navigation. However, large-area environments present challenges in terms of scalability and computational efficiency [13]. To address these difficulties, researchers have introduced context-aware multi-agent systems that improve decision-making in dynamic spaces [14]. In addition, boundary search techniques combined with online policy optimization in DRL have been shown to enhance path planning in multi-AAV navigation [15]. Unlike conventional SLAM-based methods, DRL-enabled algorithms improve autonomous AAV navigation. However, non-line-of-sight search tasks introduce additional difficulties. In [16], the AAV navigation problem in a large environment has been articulated as a Markov Decision Process (MDP) problem with sparse rewards, necessitating specialized policy control mechanisms. Similarly, [17] introduced a limited MDP-based policy with a customized reward function for rapid target localization.

Designing an optimal reward function in a sparse reward setting remains challenging. To overcome this problem, [18] proposed a DRL-based Cramer-Rao Lower Bound (CRLB) estimation to refine the reward design. Furthermore, Multi-agent Reinforcement Learning (MARL) is introduced for effective AAV path planning in large-scale environments [19]. Obstacle avoidance is critical for autonomous navigation. The authors in [20] proposed an MDP-based strategy to ensure fast convergence while avoiding collisions. However, multi-AAV navigation in large spaces often struggles with convergence speed. To address these issues, a decentralized RL-based two-stage policy framework was proposed in [21]. Despite improvements in DRL algorithms, challenges, such as reward sparsity and environmental exploration, persist. A Twin Delayed Deep Deterministic Policy Gradient (TDDDPG) mechanism was introduced in [22] to mitigate these issues. Moreover, a DRL-based cooperative navigation approach was presented in [23] to address sparse rewards in multi-AAV settings.

Recent studies have emphasized the importance of real-world adaptability. To this end, [24] suggested integrating RL algorithms with a Hardware-in-the-Loop (HITL) simulation approach, thereby significantly improving AAV navigation capabilities. Inspired by this, [25] investigated how blending DRL with the Artificial Intelligence of Things (AIoT) can result in robust AAV navigation with reduced collision risk. AAVs must retain environmental information for better decision-making. A DRL framework for AAV obstacle avoidance was proposed by [26] to solve this problem, while [27] introduces a stochastic optimization technique for trajectory planning in unidentified environments.

However, such approaches are unsuitable for GPS-denied conditions, necessitating the development of depth-based localization techniques with high Degrees of Freedom (DoF) [28]. A model-free, off-policy actor-critic framework incorporating a Deep Q-Network (DQN) was explored in [29], but its ability to adapt to partially observable spaces remains limited.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

Consequently, [30] proposed a Stochastic Valued Gradient (SVG) algorithm combined with Recurrent Neural Networks (RNNs) to improve exploratory behavior in unknown spaces. Despite these advances, large-area AAV navigation still faces decision-making uncertainties owing to its limited sensing capability. To address this, a partially observable Markov decision process framework was applied in [11] to enhance multi-AAV decision-making. Building on prior research, [13] introduced an advanced autonomous AAV navigation approach for complex environments. Recent work has predominantly focused on model-based strategies to address partial observability. The authors in [31] developed an online solution for near-optimal policy learning, integrating GRU to process sequential data. However, existing approaches struggle to provide optimal policy value approximation, preventing AAVs from efficiently reaching their target states.

Unlike baseline DRL algorithms, ARReSVG introduces several key advancements. First, it incorporates adaptive policy learning, compared to previous static policy update methods [19], [23], [30], allowing AAVs to adjust strategies dynamically to improve learning efficiency. Second, it implements a consensus path planning mechanism to enhance collaboration among AAVs for optimized navigation. Third, ARReSVG employs a GRU-enhanced actor-critic framework to efficiently approximate action-value functions and overcome scalability challenges in large and dynamic environments.

Finally, ARReSVG achieves faster convergence, outperforming previous works [19], [30] through its adaptive learning mechanism. Despite the remarkable progress in AAV navigation, DRL-enabled multi-agent system approaches still struggle with scalability, reward sparsity, and real-time adaptability in partially observable environments. To address these challenges, ARReSVG introduces an adaptive learning mechanism with GRU-based actor-critic optimization, ensuring efficient navigation in complex environments. The following sections present the ARReSVG framework and detail its architecture, implementation, and experimental evaluation.

III. System Model

The preliminary stages of the partial observability and the fundamental kinematics of AAVs in partially visible environments are described in the following subsections. This section focuses on partial observability modeling for multi-AAV navigation problems. A Partially Observable Markov Decision Process (POMDP) defines the problem scenario, and the optimization goals and constraints accompanying it are developed.

The original problem addressed in this work is to enable multi-AAVs to autonomously navigate from an arbitrary starting point to a target location in a POMDP scenario, while avoiding obstacles in complex environments. The goal is to maximize the probability of successful navigation and minimize the collision risk and trajectory drift rate.

A. Preliminaries: POMDP

Generally, Markov Decision Process (MDP) provides a mathematical framework for decision-making under uncertainty. The primary objective of MDP is to compute an optimal policy π^* that maximizes the expected cumulative reward [11] over an infinite horizon.

$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t))\right] \tag{1}$$

In contrast, POMDP extends MDP to scenarios where the agent cannot fully observe the system state. A POMDP [11] is formulated as a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{O}, P, R, Z, \Omega \rangle$, where $\mathcal{O}$ is the set of observations, Z is the observation probability function, Ω represents the initial belief state distribution, $\mathcal{S}$ as states, P depicts the conditional probability distribution, $\mathcal{A}$ is action, and R is the reward. The observation probability $Z(o \mid s, a)$ specifies the likelihood of observing o given state s and action a. The belief states $\hat{b}_t$, a probability distribution over states, is updated using Bayes' rule [8], [13] as

$$\hat{b}_{t+1}(s') = \frac{Z(o_{t+1} \mid s', a_t) \sum_{s \in \mathcal{S}} P(s' \mid s, a_t) \hat{b}_t(s)}{\sum_{s'' \in \mathcal{S}} Z(o_{t+1} \mid s'', a_t) \sum_{s' \in \mathcal{S}} P(s'' \mid s', a_t) \hat{b}_t(s')} \tag{2}$$

The key objective of POMDP is to compute a policy π^* that maximizes the expected cumulative rewards over an infinite horizon, given partial observability [11];

$$\pi^* = \arg\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(\hat{b}_t))\right] \tag{3}$$

In (3), $\hat{b}_t$ represents the belief state at time t.

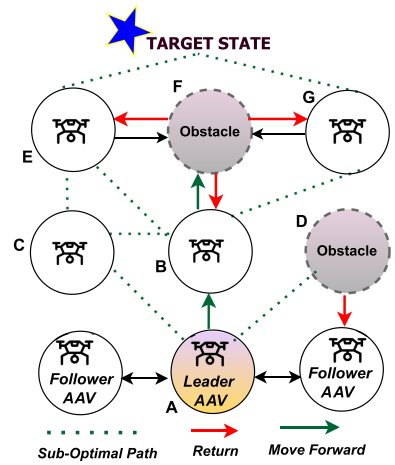

Fig. 1. Multi-AAV navigation under partial observability.

B. Partial Observability Scenario for Multi-AAV Navigation

A general view of the leader-follower method utilized by a multi-AAV navigation model in a partially visible environment is shown in Fig. 1. The leader AAV begins its mission at point A, navigating toward the target state (represented by the star) while avoiding obstacles. The environment features contain obstacles like buildings and terrain (represented in shaded regions) and require the AAV to make real-time decisions based on its limited sensing capabilities. Movement paths are shown with green

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

arrows for forward moves, red arrows for return movements, and dotted green paths indicating sub-optimal or oscillatory routes caused by memory constraints and limited perception.

Consider that the leader AAV explores potential paths by sensing the environment, deciding on the optimal direction, and attempting to avoid deadlocks caused by repeated oscillations between points A and the target state. Once the leader AAV identifies a sub-optimal path, the follower AAVs proceed along the same trajectory to reach the target. The follower AAVs rely on inter-AAV communication to adapt to changes in the leader's path but do not independently sense the environment. The success criteria in this design are achieved when all AAVs (leader and followers) reach the target state within the constraints of the environment without collisions and when navigation is completed while optimizing the cumulative reward cost. Meanwhile, the failure criteria occur if the leader AAV becomes trapped in a deadlock or fails to find a possible route. Also, when one or more AAVs collide with obstacles and the multi-AAVs cannot complete the tasks within the iterations, it leads to the failure criteria.

This model is framed within a POMDP, where states include AAV positions, environmental features, actions involving directional movement and observations, positional information, obstacle proximity, and IMU data come from the AAV's sensors. The system optimizes rewards for reaching the target efficiently while minimizing penalties for collisions and deviations from the optimal path. In addition, Table I highlights the list of notations used in this paper and their descriptions.

TABLE I
LIST OF NOTATIONS

Notation	Description
η	Learning rate
Γ	Policy function
ϕ	Tuning parameter to optimize
Θ^{ν}	Probability distribution
θ	Parameters controlling the distribution
ξ	Parameter defines the policy
χ^{l}	Discount factor
o_t	Observation
R_{total}	Total expected cumulative reward
$\mathbb{E}$	Expectation operator
π	Policy function
$\mathcal{H}_k$	Past observations
γ	Discount factor
τ	Trajectory
s_t	AAV state
u_t	Input control
v_t	Linear velocity
$d_{\min}$	Minimum safe distance
β	Momentum coefficient
$R(s, a_t)$	Reward function
$P(s' \mid s, a_t)$	State transition probability function
$Z(o \mid s, a_t)$	Observation function probability
$\mathcal{Q}^{\xi}(v_k, \alpha_k)$	Action-value function
$\nabla_{\theta} L(\theta_t)$	Loss function gradient
$\varrho(v_{k+l}, \alpha_{k+l})$	Reward w.r.t. action-value function
$\mathcal{V}_{\xi}^{\mathcal{H}}(v_k)$	Value function for state v_k
$\frac{\partial J(\theta)}{\partial \theta}$	Objective function gradient
$\partial \mu^{\theta}(h_t)$	Policy gradient

C. Problem Formulation and Kinematics of AAV

The key objective of a multi-AAV system is to optimize cooperative navigation in a partially observable environment by ensuring collision-free trajectories, minimizing trajectory drift, and maximizing the success rate of reaching a target destination. This optimization problem is formulated within the framework of a Partially Observable Markov Decision Process (POMDP) [13]. The objective is to compute an optimal policy π^* that maximizes the expected cumulative rewards over an infinite horizon, and it can be expressed as shown in (4).

$$\pi^* = \underset{\pi}{\operatorname{argmax}}\, \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t)\right] \tag{4}$$

The key constraints are defined below,

$d_{\text{safe}} > \|\mathbf{p}_{\text{AAV}_i} - \mathbf{p}_{\text{AAV}_j}\|, \quad \forall i \neq j;$

$a_t \in \mathcal{A}, \quad \forall t;$

$o_t = \mathcal{O}(s_t),$

The reward function $R(s_t, a_t)$ is introduced to help the AAVs achieve efficient and collision-free navigation and is defined as follows:

$$R(s_t, a_t) = \begin{cases} +1, \text{AAV moves closer to the target safely;} \\ 0, \text{AAV remains idle;} \\ -0.5, \text{AAV performs any oscillatory behavior;} \\ -1, \text{AAV collides with obstacle or other AAV;} \end{cases}$$

From (4), γ is the discount factor that assures cumulative rewards, $R(s_t, a_t)$ is the reward function that quantifies navigation efficiency and collision avoidance, s_t is the state concerning time t, and a_t is the actions. The reward function penalizes collisions, oscillations, and deviations from the optimal path while encouraging progress toward the target destination. For the constraints the $\mathbf{p}_{\text{AAV}_i}$ and $\mathbf{p}_{\text{AAV}_j}$ represents the positions of AAVs i and j, and d_{safe} is the minimum safety distance that must be maintained. $\mathcal{A}$ is the set of possible actions that each AAV can execute at any time t, and the o_t represents the observation to time t, which is a subset of the true state s_t. Here, $\mathcal{O}$ is the observation function which combines the actual state with the observed state.

Generally, AAV state with respect to time t is represented in $s_t = [x_t, y_t, z_t, \theta_{xt}, \theta_{yt}, \theta_{zt}]$, where $x_t, y_t, z_t \in \mathbb{R}$ denote the AAV's Cartesian position, and $\theta_{xt}, \theta_{yt}, \theta_{zt} \in [0, 2\pi]$ represent its orientation in Euler angles. The state evolves according to the kinematics model $s_{t+1} = f(s_t, u_t) + w_t$, where $u_t = [v_t, \omega_{xt}, \omega_{yt}, \omega_{zt}]$ is the control input, $f(\cdot)$ is the motion model of the AAV, and w_t represents process noise. The control inputs are bounded by physical constraints such that $v_t \in [v_{\min}, v_{\max}]$,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

where v_{min} and v_{max} are the minimum and maximum allowable velocities, and the angular velocities $\omega_{xt}, \omega_{yt}, \omega_{zt} \in [-\omega_{\text{max}}, \omega_{\text{max}}]$, where ω_{max} is the maximum allowable angular velocity.

To ensure collision-free navigation, the AAV must maintain a safe distance from obstacles expressed as $d(u_t, \text{obstacle}) > d_{\text{safe}}$, where d_{safe} is the minimum allowable distance to any obstacle. Additionally, in the multi-AAV system, each AAV must maintain safe separation from other AAVs to avoid inter-AAV collisions. This constraint is expressed as $d_{\text{follower},i,j}(t) > d_{\text{min}}$, where d_{min} is the minimum distance required between AAVs i and j. During navigation, the AAV detects any repeated oscillatory movements between waypoints, which trigger an exploratory action by sampling a random direction within its sensing range to avoid a deadlock. Moreover, the target state is within the AAV's limited sensing range, and the AAV prioritizes the forward movement, provided the path is obstacle-free and navigates toward the target state.

The AAV's observations are limited by sensor capabilities. The observation at time t is modeled as $o_t = h(s_t, u_t) + v_t$, where $h(\cdot)$ represents the sensor model and v_t accounts for observation noise [31]. Also, the belief states $\hat{b}_t$, incorporating sensor observations and the motion model to refine the AAV's estimate of the environment. The decision-making process of the leader AAV is based on its mounted sensors and the trained DRL policy to make accurate navigational decisions with partial observability. The decision-making process depends on local observation, action selection, memory constraints, and limited sensing.

In the multi-AAV scenario, the leader AAV operates in a partially observable environment (it only has access to local observations). Due to limited sensing ranges, AAVs may not identify distant obstacles, leading to the choice of suboptimal decisions and revisiting already explored paths. In addition, the leader AAV utilizes a GRU-based policy network with memory and information regarding past states and observations. This helps AAVs to make better-informed decisions. In local observation, the leader of the AAV collects environmental information initially, and based on local observations, the leader evaluates a set of possible actions using its policy network. The output of action selection results in an optimal action for the current state. Action selection balances obstacle avoidance, enables forward movement to the target state, and minimizes the cumulative reward cost.

The optimization objectives for the leader AAV focus on minimizing the total path length L, defined as $L = \sum_{t=0}^{T} \|s_{t+1} - s_t\|$ while avoiding collisions and maximizing efficiency. The leader's role includes path exploration and determining a feasible trajectory for the follower AAVs. The follower AAVs aim to track the leader's trajectory while maintaining safe separation and adapting to any changes in the leader's path.

Several measures are used to evaluate the performance of the proposed system. The percentage of flights in which every AAV reaches its target safely and without collision is measured by the success rate. The percentage of trajectories that collided with obstacles or other AAVs can be measured using the collision rate. The waypoint distance measures the overall distance traveled between consecutive waypoints, indicating trajectory efficiency, whereas trajectory drift shows how far the trajectory of the AAV drifts from its optimal path. Finally, cumulative awards, which capture task completion, safety, and navigation efficiency, evaluate the overall rewards earned throughout the operation.

IV. ARReSVG

This section discusses the impact of the adaptive learning mechanism on a rapid, recurrent stochastic valued gradient algorithm. It also describes how the ARReSVG framework converges with the aid of DRL and GRU networks for effective autonomous navigation in complex environments.

As depicted in Fig. 2, the proposed ARReSVG framework integrates a rapid recurrent stochastic policy gradient with a GRU Net, which takes the multi-AAV navigational parameters (sensory data and IMU) as inputs and produces a set of optimal actions as outputs, thereby helping the AAVs to reach the target state. The navigational inputs are initially passed to the actor module in the actor-critic unit. Simultaneously, these inputs undergo random sampling, storing the information in the pool. Then, feedback-enabled batch sampling is performed, and the results are fed to the action-value estimation approximation unit, which is empowered with a GRU Net that helps remember previous observations and enables agents to estimate the current state more accurately. The recurrent target network in the actor module calculates the best action to take in a given state and collects and validates the rewards with the help of the associated recurrent policy network unit. This ability helps to maximize the expected cumulative rewards, resulting in better navigation over time. The critic module evaluates how well the actor-network selects the actions and estimates the state value function based on feedback inputs over time. This triggers the actor module to make better decisions by providing feedback regarding the chosen actions.

A. DRL for Adaptive Learning-based RReSVG (ARReSVG)

This section proposes a DRL-empowered adaptive learning-based rapid recurrent policy gradient technique for addressing the partial observability problem for the multi-AAV. Each AAV in the multi-AAV scenario utilizes the ARReSVG framework to process the observations $\mathcal{O}$ and make optimal navigation decisions by sharing the state information with neighboring AAVs in the environment and helping to reach the target state. Algorithm 1 shows the working flow of the proposed ARReSVG framework. In POMDP modeling, the action-valued function is defined as $Q^{\pi}(s, a_t)$, which gives the expected cumulative reward of acting a in the state s under a policy π plays a major role in policy updates. Since the states are not fully observable, agents must rely on observations to make decisions. $Q^{\pi}(s, a_t)$ accounts for this uncertainty by considering both the immediate reward r and the anticipated future rewards r', considering the environment's stochastic nature and partial observability.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

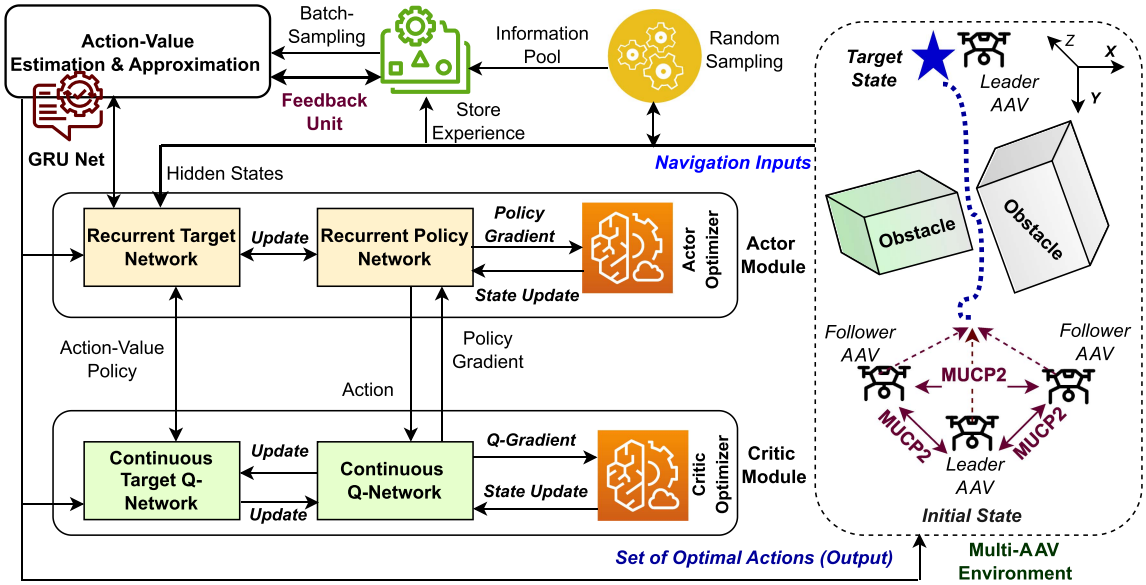

Fig. 2. Proposed ARReSVG framework for the autonomous navigation of multi-AAV in partially observability.

The POMDP's action-valued function is represented as,

$$Q^{\pi}(s,a_t) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\lambda^t r_t \mid s_0 = s, a_0 = a_t\right] \tag{5}$$

where $\mathbb{E}_{\pi}$ denotes the expectation under the policy π and the λ represents the discount rate that depicts the value of the future reward, r_t is an immediate reward at time t, and the sum extends over all future time steps. From (5), the expectation function considers the uncertainty in the transitions between states and the observations received. Policy updates in POMDP often involve improving the current policy based on the estimated action values. The expectation with existing policy Q is drawn from the real action-value function [30], which is depicted in (5).

$$\frac{\partial \mathcal{L}(\phi)}{\partial \phi} = \mathbb{E}_{\upsilon} \sim \Theta^{\nu}\left[\frac{\partial \Gamma^{\nu}(\upsilon,\alpha)}{\partial \alpha}\bigg|_{\alpha=\Gamma^{\phi}(\upsilon)} \frac{\partial \Gamma^{\phi}(\upsilon)}{\partial \phi}\right] \tag{6}$$

where $\mathcal{L}(\phi)$ represents the loss function depends on ϕ, parameters to be optimized. Θ^{ν}, represents the distribution from the sample υ is drawn. $\Gamma^{\phi}(\upsilon)$, gives a policy function parameterized by ϕ w.r.t to calculate the gradient $(\Gamma^{\nu}(\upsilon,\alpha))$. In Algorithm 1, the general policy notation Γ_{ϕ} is represented as π_{θ}, where π denotes the stochastic policy and θ are the learnable parameters. Both policy notations refer to the same concept, but π_{θ} is considered in the algorithm implementation for consistency with actor-critic frameworks.

For (4), with the finite horizon Bellman equation, the value of a state S, at a particular time t step in terms of rewards r and r'.

$$V_t^*(s) = \max_u \left[R(s,u) + \gamma \sum_{u'} \eta(u'|s,u) V_{t-1}^*(s')\right] \tag{7}$$

where $V_t^*(s)$ represents the optimal value function at time t as the maximum cumulative rewards r by taking the best action in the current state. η represents the learning rate parameter, controls the step size, and the $R(s,u)$ highlights the immediate reward obtained for performing a suitable action. From (6), the finite horizon POMDP for an arbitrary policy is identified.

$$V_t^{\pi}(s) = R(s,\pi_t(s)) + \sum_{\omega} \eta(\omega|s,\pi_t(x)) V_{t-1}^{\pi}(\tau(s,\pi_t(s),\omega)) \tag{8}$$

Combining (5) & (6), the finite horizon action-value function for POMDP is formulated and expressed below.

$$Q_t^*(s_t,u_t) = R(s',u') + \gamma \sum_{u'} \eta(u'|s,u) V_{t-1}^*(s') \tag{9}$$

The error bound for (8) is computed to obtain the absolute error over the time step of the AAV at t steps as shown in (9).

$$\sup_x |V_t^*(s) - V^*(s)| \leq \frac{\gamma\epsilon}{1-\gamma_t} \tag{10}$$

where $V_t^*(s)$, $V^*(s)$ are the true optimal and estimated errors, and their value is to be approximated at most to a precision. The recurrent policy update in the Algorithm 1 has been updated using (10).

$$\frac{\partial \mathcal{L}(\phi)}{\partial \phi} = \mathbb{E}_{\zeta}\left[\sum_k \omega^{k-1} \frac{\partial \Gamma^{\nu}(\upsilon_k,\alpha)}{\partial \alpha}\bigg|_{\alpha=\Gamma^{\phi}(\upsilon_k)} \frac{\partial \Gamma^{\phi}(\upsilon_k)}{\partial \phi}\right] \tag{11}$$

where $\frac{\partial \Gamma^{\nu}(\upsilon_k,\alpha)}{\partial \alpha}$, ensures the back-propagation of the policy in the valued gradient function.

B. Value Approximation Using Gated Recurrent Unit (GRU) Network in ARReSVG

In a partial observability setting, the multi-AAV operates in an environment without direct observation of the true underlying state. Instead, it receives observations that are probabilistically dependent on the state variable S. Based on the history of observations and actions, the agent performs decisions based on its belief state [8]. The value function is defined as $V(\hat{b}_t)$,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Adaptive-Learning Based Rapid Recurrent Stochastic Valued Gradient Algorithm (ARReSVG).

Input: Input sequence of observations O, Initial hidden state h_0
Output: Value estimates for each time step
1: **Initialization**: Initialize critic network $K_t^{\kappa}(a_t, h_t)$ and actor $\pi_t^{\theta}(h_t)$ with w and θ, continuous target critic networks $K_t^{\kappa'}(a_t, h_t)$ and recurrent target actor network $\pi_t^{\theta'}(h_t)$ with $w', \kappa', \theta', \theta$.
2: Declare experience buffer R
3: **for** episodes $= 1$ to M **do**
4: **Episode Initialization**:
5: Declare a stochastic random process N for a exploration
6: Get initial observation O_0 (or history trajectory h_0).
7: **for** $t = 1$ to T **do**
8: **Agent's Interaction**:
9: Perform execution of $a_t = \pi^{\theta}(h_t, \nu)$ with $\nu \sim \beta$ (adaptive stochastic operation).
10: Score reward r_t and observation value O_t.
11: Store S transition (h_{t-1}, a_t, o_t, r_t) into R
12: Modify history trajectory information $h_t = [h_{t-1}, o_t, a_t]$.
13: **Adaptive Learning**: Get feedback inputs over time T while considering action A & reward R estimation
14: $v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot \nabla_{\theta} L(\theta_t) + \gamma \cdot F_t$
15: $\theta_{t+1} = \theta_t - \eta \cdot v_t$
16: Update information pool buffer (*Go to Step 13*)
17: Perform mini-batch sampling of N transitions $\{(h_{i-1}, o_i, a_i, r_i)\}_{i=1}^{L}$ from R.
18: Compute target $y_t^i = r_t^i + \gamma K^{\kappa'}(h_{t+1}^i, \pi^{\theta'}(h_{t+1}^i, \nu))$
19: Compute critic update (w.r.t. Back-propagation through T) $\nu \sim \beta$.
20: $\Delta\kappa = \frac{1}{NT} \sum_i \sum_t (y_t^i - K^{\kappa}(h_t^i, a_t^i)) \frac{\partial K^{\kappa}(h_t^i, a_t^i)}{\partial \kappa}$
21: Compute actor update (w.r.t. Back-propagation through T)
22: $\Delta\theta = \frac{1}{NT} \sum_i \sum_t \frac{\partial K^{\kappa}(h_t^i, \pi^{\theta}(h_t^i, \nu))}{\partial a}\Big|_{\pi^{\theta}(ht^i, \nu)} \frac{\partial \pi^{\theta}(h_t^i, \nu)}{\partial \theta}$ with $\nu \sim \beta$
23: Update the target networks:
24: $\kappa'\prime \leftarrow \tau\kappa + (1-\tau_t)\kappa'$
25: $\theta'\prime \leftarrow \tau\theta + (1-\tau_t)\theta'$
26: **end for**
27: **end for**
28: **return** V

representing the anticipated cumulative reward when starting from a belief state $\hat{b}_t$ and following an optimal policy. In our work, we utilized the variant of RNNs, Gated Recurrent Unit, which is used to model sequential data, and dependencies over time are depicted in Fig. 3. The utilized GRU Net is a variant of

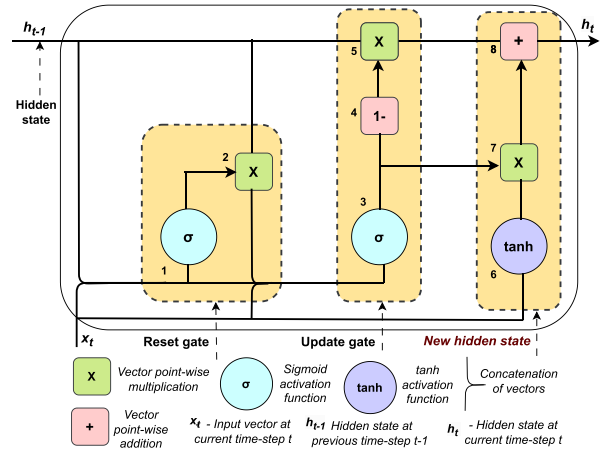

Fig. 3. Gated recurrent unit network model.

RNN [12] that can capture long-term dependencies in observation sequences [32].

In addition, GRU-Net comprises reset and update gates. These smaller numbers of gates help reduce the number of parameters in GRU-Net so that they can perform computationally faster than other RNN algorithms, such as Long Short Term Memory (LSTM) and Bi-directional RNN (Bi-RNN).

$$\frac{\partial J(\theta)}{\partial \theta} = \mathbb{E}_{\tau}\left[\sum_t \eta^{t-1} \frac{\partial Q^{\mu}(h_t, a)}{\partial a}\Big|_{a=\mu^{\theta}(h_t)} \frac{\partial \mu^{\theta}(h_t)}{\partial \theta}\right] \tag{12}$$

where $\frac{\partial J(\theta)}{\partial \theta}$ is the gradient of the objective function [8], [11], $\partial\mu^{\theta}(h_t)$ is the policy gradient, $\mathbb{E}$ depicts the expectation operator, and τ represents the trajectory. (11) highlights the update of the stochastic gradient policy. Once the state information is updated, it is loaded into the replay buffer R.

$$v_t = \beta \cdot v_{t-1} + (1-\beta) \cdot \nabla_{\theta} L(\theta_t) + \gamma \cdot F_t \tag{13}$$

$$\theta_{t+1} = \theta_t - \eta \cdot v_t \tag{14}$$

where β is the momentum coefficient, v_t is the linear velocity at time t, and $\nabla_{\theta} L(\theta_t)$, is the loss function gradient. From (12) & (13), adaptive learning is applied to (10) and (11) to obtain continuous feedback from the observations, which helps to enhance the optimal trajectory decision-making more quickly.

$$\mathcal{V}^{\xi}(\upsilon_k) = \mathbb{E}\left[\sum_{l=0}^{\infty} \chi^l \varrho(\upsilon_{k+l}, \alpha_{k+l}) | \upsilon_k\right] \tag{15}$$

$$\mathcal{V}_{\xi}^{\mathcal{H}}(\upsilon_k) = \mathbb{E}\left[\sum_{m=1}^{\infty} \chi^{m-1} \varrho_{k+m} | \upsilon_k, \mathcal{H}_k, \xi\right] \tag{16}$$

Applying (13) & (14) on (15) gives the updated value function $\mathcal{V}^{\xi}(\upsilon_k)$ of the state after updating the policy in (8) as shown in (16). ξ, is a parameter defines the policy, $\mathcal{H}$, history observations, and $\varrho(\upsilon_{k+l}, \alpha_{k+l})$ is the reward w.r.t the action-value function.

$$\mathcal{Q}^{\xi}(\upsilon_k, \alpha_k) = \mathbb{E}\left[\sum_{l=0}^{\infty} \chi^l \varrho(\upsilon_{k+l}, \alpha_{k+l}) | \upsilon_k, \alpha_k\right] \tag{17}$$

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

$$\mathcal{Q}_{\xi}^{\mathcal{H}}(\upsilon_k, \alpha_k) = \mathbb{E}\left[\sum_{m=1}^{\infty} \chi^{m-1} \varrho_{k+m} | \upsilon_k, \mathcal{H}_k, \alpha_k, \xi\right] \tag{18}$$

From (17) and (18), the reward function is computed by taking action on h_t and is stated as follows.

$$\mathcal{R}_{\xi}^{\mathcal{H}}(\upsilon_k, \alpha_k) = \mathbb{E}[\varrho_{k+1} | \upsilon_k, \mathcal{H}_k, \alpha_k, \xi] \tag{19}$$

Hence, the action-valued function in POMDP encapsulates the expected cumulative rewards of taking actions in partially observable environments, providing a foundation for policy updates and decision-making under uncertainty. As highlighted in Fig. 1, the limited sensing issue is that the leader AAV may not identify distant obstacles. This features the leader AAV in identifying suboptimal paths by exploring past explored points or choosing adjacent paths to move forward. To overcome the challenges in path exploration, the leader AAV utilizes the GRU-Net to maintain the history of past states and observations to make better navigational decisions.

Now, let's consider the belief state $\hat{b}_t$ to be concerning the observations and actions up to that point, and it is updated using the POMDP lemma as mentioned in (20).

$$\hat{b}_{t+1} = P(s_{t+1} | a_t, o_{t+1}, \hat{b}_t) \tag{20}$$

$$h_t = \text{GRU Net}(\hat{b}_t, h_{t-1}) \tag{21}$$

where h_t represents a hidden state of the GRU Net. The belief state $\hat{b}_t$ [8] is fed into the GRU at each time step. The GRU Net modifies its internal state as per the present input and the past state, as mentioned in (21). The value approximation of the GRU Net is illustrated in Algorithm 2. The result of the GRU-Net at each time is used for value approximation $\frac{\partial J(\theta)}{\partial \theta}$ as given in (12). This output can be denoted as υ_t, representing the estimated value at time t. Finally, the overall value function is approximated as the sum of the discounted future values [33].

$$V(\hat{b}_t) \approx \sum_{k=0}^{\infty} \lambda^k \upsilon_{t+k} \tag{22}$$

In (22), the λ represents the discount rate. The parameters of the GRU Net are trained using back-propagation through a time algorithm. The training minimizes the difference between predicted values υ_t and the returns obtained during learning. Thus, the employed GRU Net contributes significantly to approximating trajectories in the POMDP scenario during optimization.

Algorithm 2: Value Approximation Using GRU Net-Based ARReSVG.

Input: Training data: (s_t, a_t, r_t, s_{t+1}) for each time t
Output: Trained GRU action value parameters
Initialize GRU Net parameters: $W_r, U_r, b_r, W_z, U_z, b_z, W_h, U_h, b_h, W_o, b_o$, Initialize target network parameters: $\theta_{\text{target}} \leftarrow \text{Copy}(\theta)$, replay buffer B: Replay buffer, Discount rate: λ, Exploration rate parameters: $\epsilon, \epsilon_{\text{min}}, \epsilon_{\text{decay}}$, Optimization algorithm parameters.
Declare GRU Net hidden state: h_0
for episode $= 1$ to num_episodes **do**
  Reset to the initial state: s_0
  for $t = 1$ to max_steps **do**
    Exploration-exploitation trade-off
    if with probability ϵ **then**
      $a_t \leftarrow$ RandomAction()
    else
      $a_t \leftarrow$ SelectAction(s_t) using the current policy
    end if
    Perform execution of the action and observe the next state and reward
    Perform action a_t, observe reward r_t and move next state s_{t+1}
    Store state transitions (s_t, a_t, r_t, s_{t+1}) in Replay buffer R
    Sampling a mini-batch from replay buffer R
    Sample mini-batch B from Replay buffer:
    Compute target Q-values
    for each (s_i, a_i, r_i, s_{i+1}) in B **do**
      if s_{i+1} is terminal **then**
        $target_i \leftarrow r_i$
      else
        $target_i \leftarrow r_i + \gamma \cdot \max_{a'} Q_{\theta_{\text{target}}}(s_{i+1}, a')$
      end if
    end for
    Update GRU parameters using back-propagation
    Compute action values $Q_\theta(s_i, a_i)$ using the GRU
    Compute loss: $\mathcal{L} \leftarrow \sum_i (Q_\theta(s_i, a_i) - target_i)^2$
    Update θ using optimization algorithm
    Update target network parameters periodically
    if stepmod target_update_frequency $= 0$
    $\theta_{\text{target}} \leftarrow \text{Copy}(\theta)$
    end if
    Compute decay exploration rate
    $\epsilon \leftarrow \max(\epsilon_{\text{min}}, \epsilon \cdot \epsilon_{\text{decay}})$
  end for
end for

C. Multi-AAV Consensus Path Planning (MUCP2) Mechanism in ARReSVG

Significant considerations in the proposed ARReSVG framework are the effects and setup of multi-AAV navigation. The virtual environment in which navigation is given is an essential component of this. By employing a distributed approach, the framework allows AAVs to communicate and jointly identify routes that avoid collisions while maintaining formation and avoiding obstacles. To guarantee effective navigation, the consensus path planning system makes use of dynamic information exchange between AAVs. While operating independently, each AAV shares critical parameters to improve cooperation and flexibility. The leader AAV can start its navigation operation from any location at first.

During navigation, the leader AAV, denoted as L, initiates the navigation mission by sharing its local information with neighboring AAVs. This information includes state parameters,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

position $\mathbf{p}_L = [x_L, y_L, z_L]$, velocity $\mathbf{v}_L$, acceleration $\mathbf{a}_L$, orientation, and environmental states, $\mathbf{o}_L$ and $\mathbf{e}_L$, respectively.

The follower AAVs, denoted as F_i, where $i \in \{1, 2, \ldots, N\}$, share their own states $\mathbf{p}_i$, $\mathbf{v}_i$, $\mathbf{a}_i$, and local observations $\mathbf{o}_i$. This data is aggregated into a global information pool $\mathcal{I}_t$ at time t, which serves as the basis for trajectory updates. The MUCP2 mechanism employs an iterative consensus process to update the trajectories of AAVs. At each iteration k, the trajectory will be updated. Each AAV $u \in \{L, F_1, F_2, \ldots, F_N\}$ updates its trajectory plan $\boldsymbol{\tau}_u^{k+1}$ based on its state and the information from its neighbor AAVs,

$$\boldsymbol{\tau}_u^{k+1} = \boldsymbol{\tau}_u^k + \eta \sum_{v \in \mathcal{N}(u)} w_{uv} \left(\boldsymbol{\tau}_v^k - \boldsymbol{\tau}_u^k\right) \tag{23}$$

where η is the learning rate [30], $\mathcal{N}(u)$ represents the collection of neighboring AAVs, and w_{uv} is the weight reflecting the reliability of information exchange between AAVs u and v. Also, each AAV evaluates the collision risk using a potential function $\phi(\mathbf{p}_u, \mathbf{p}_v)$ for any pair (u, v), where:

$$\phi(\mathbf{p}_u, \mathbf{p}_v) = \begin{cases} \infty, & \text{if } \|\mathbf{p}_u - \mathbf{p}_v\| < d_{\text{min}}, \\ 0, & \text{if } \|\mathbf{p}_u - \mathbf{p}_v\| \geq d_{\text{safe}}, \end{cases}$$

with d_{min} and d_{safe} representing the minimum and safe separation distances, respectively.

Moreover, each AAV computes the action-value $Q(\mathbf{s}_u, \mathbf{a}_u)$ using a GRU-Net to determine the optimal action $\mathbf{a}_u$ under partial observability. The state $\mathbf{s}_u$ comprises the AAV's position, velocity, orientation, and local observations;

$$\mathbf{a}_u = \arg\max_{\mathbf{a}_u} Q(\mathbf{s}_u, \mathbf{a}_u) \tag{24}$$

The MUCP2 mechanism performs consensus iteration and continues until the AAVs collectively converge on a set of collision-free trajectories $\{\boldsymbol{\tau}_L, \boldsymbol{\tau}_{F_1}, \ldots, \boldsymbol{\tau}_{F_N}\}$. The leader AAV communicates crucial trajectory updates and ensures global path consistency at convergence. To satisfy sense-and-avoid requirements, the following AAVs maintain formation and fine-tune their local trajectories. In addition to addressing challenges during uncertain situations with intricate settings and limited sensing capabilities, this cooperative decision-making process guarantees safe navigation. The MUCP2 mechanism is adaptable due to its distributed structure, which guarantees successful navigation in a challenging environment.

V. Results and Discussions

A. Experimental Setup

The Ardupilot SITL platform with Gazebo World interface integration simulates the numerical evaluation of autonomous multi-AAV navigation. In addition, the simulation platform is integrated into an OpenAI Gym environment for DRL training. The entire multi-AAV simulation and DRL training were conducted on an Intel Core i7 CPU processor with 2 TB of memory enabled on 32 GB of RAM, with the addition of an NVIDIA GeForce GTX. A large-scale 3D virtual environment was initially created on the Gazebo platform using 3 km^2. The continuous space of the navigation area was split into four environments based on obstacle intensity and terrain (Env. 1: large area with obstacle intensity of 60% to 65%, Env. 2: Large area with an obstacle intensity range of 65% to 70%, Env. 3: Large-scale scattered environment with an obstacle intensity of up to 75%, and Env. 4: Unknown environment with dense obstacles).

TABLE II
Simulation Parameters

Parameter (Units)	Value
Virtual environment (km^2)	3
AAV's initial speed (m/s)	0
Number of AAVs	6
Maximum speed (m/s)	10
Flight altitude (ft.)	150
Learning rate	0.001
Smoothening rate factor	0.99
Batch size (samples)	64
Replay buffer size (samples)	10000
Hidden states (neurons)	64
GRU time to store information (time steps)	10
Target network learning stability (steps)	500
Area coverage reward	0 or 1
Number of training episodes	1000

The major simulation parameters used in this work are shown in Table II. In the multi-AAV setup, the maximum number of participating AAVs in the navigation is limited to 6, and each AAV's initial speed was limited to 0 m/s, the maximum speed was configured to 10 m/s, and it started navigating over varying terrain at an altitude of 150 ft. Custom adjustments were made to the sensor parameters and waypoints to perform the navigation and reach the target state efficiently. Finally, the Gym environment class defines the AAV model and navigates to accomplish the maximum optimal rewards for carrying out the mission.

B. Result Analysis of ARReSVG Algorithm

This subsection discusses the analysis of the experimental results of the proposed ARReSVG algorithm. The ARReSVG framework with several baseline DRL algorithms was employed for autonomous multi-AAV navigation, and the safe navigation capability observed with partial observability was evaluated. Table III compares the proposed ARReSVG performance with other DRL algorithms for multi-AAVs under partial observability. The ARReSVG algorithm is compared with baseline DRL algorithms, Forward Reinforcement Stochastic Value Gradient (FRSVG) [30], Deep Deterministic Policy Gradient (DDPG) [13], [35], Twin Delayed Deep Deterministic Policy Gradient (TD3) [34], and Recurrent Stochastic Value Gradient (RSVG) [30]. Fig. 4(a) shows the obstacle intensity in conjunction with the navigation success rate in partially observable spaces.

ARReSVG achieved a 94% higher navigation success rate compared to the RSVG, DDPG, FRSVG, and TD3 algorithms. Fig. 4(b) depicts the safe navigation capability of the proposed

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

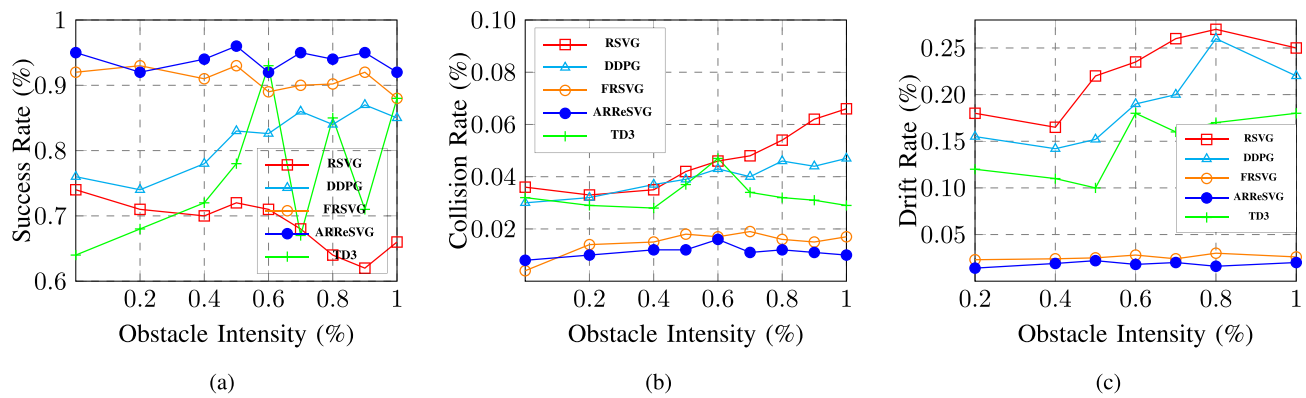

Fig. 4. Multi-AAV safe navigation and collision avoidance capability under partial observability: (a) success rate vs. obstacle intensity, (b) comparison between collision rate and obstacle intensity, and (c) drift rate vs. obstacle intensity.

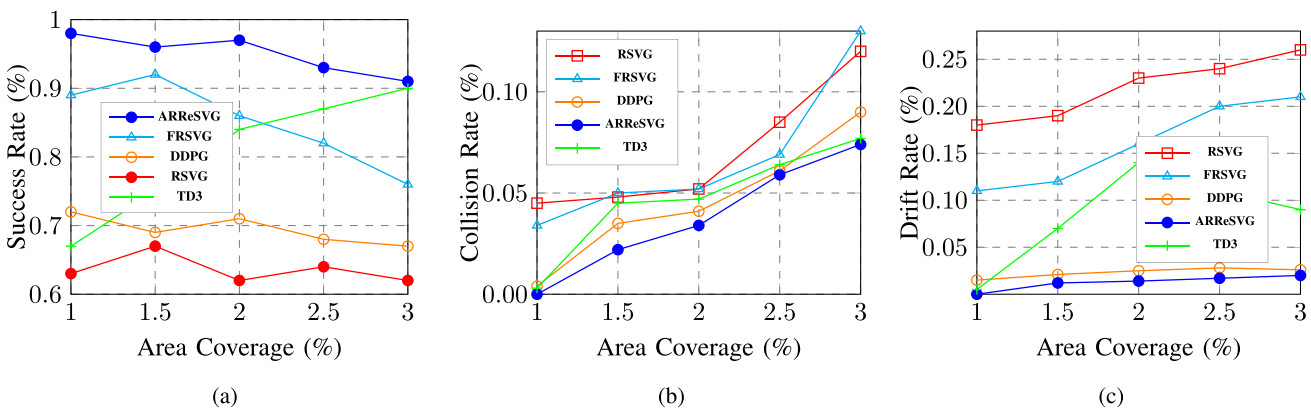

Fig. 5. Comparison of the ARReSVG with other DRL algorithms using multi-AAV navigation in complex environments: (a) evaluating success rate in area coverage, (b) assessing collision rate in area coverage, and (c) comparing drift rate with area coverage.

TABLE III
COMPARISON OF ARRESVG WITH OTHER DRL ALGORITHMS

DRL Method	Success Rate (%)	Collision Rate (%)	Drift Rate (%)	Way-point Distance (m)
RSVG [30]	76	0.038	0.17	720
DDPG [13]	77	0.03	0.15	600
FRSVG [30]	92	0.02	0.03	470
ARReSVG	**94**	**0.01**	**0.02**	**400**
TD3 [34]	93	0.03	0.11	560

framework compared with the collision rate and obstacle intensity. The collision rate of ARReSVG was comparatively lower than that of the other DRL algorithms. However, the FRSVG algorithm also provided a lower collision rate. Furthermore, Fig. 4(c) shows the deviation rate of the AAVs with obstacle intensity in the ARReSVG framework. In addition, Fig. 5 illustrates a comparison of the success, collision, and drift rates against the corresponding area coverage utilized in the navigation task. Fig. 5(a) depicts the comparison between the success rate and area coverage; the ARReSVG algorithm resulted in 95% and thus achieved a balanced path efficiency.

Compared with all other DRL algorithms utilized, the proposed ARReSVG significantly outperforms and achieves forward exploration of state action pairs of FRSVG, deterministic policy update in continuous action spaces of DDPG, continuous control mechanisms of TD3, and limited capability of learning from observation sequences of RSVG. Fig. 5(b) shows the collision rate against the area coverage, and Fig. 5(c) shows a comparison between the drift rate and area coverage. Among the baseline algorithms used, the ARReSVG algorithm achieved the best performance. Additionally, the safe navigation of multi-AAVs by addressing the waypoint distance and obstacle intensity is highlighted in Fig. 6. Fig. 6(a) shows the waypoint distance against obstacle intensity.

ARReSVG has a minimum waypoint distance. Similarly, Fig. 6(b) shows a comparison between waypoint distance and area coverage. In the ARReSVG framework, waypoint distances are dynamically adjusted based on sensing and environmental constraints, ensuring that the system operates optimally in the chosen environments. From the experimental study, DDPG and TD3 were better in continuous action spaces but lacked mechanisms for handling partial observability. RSVG addresses partial

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

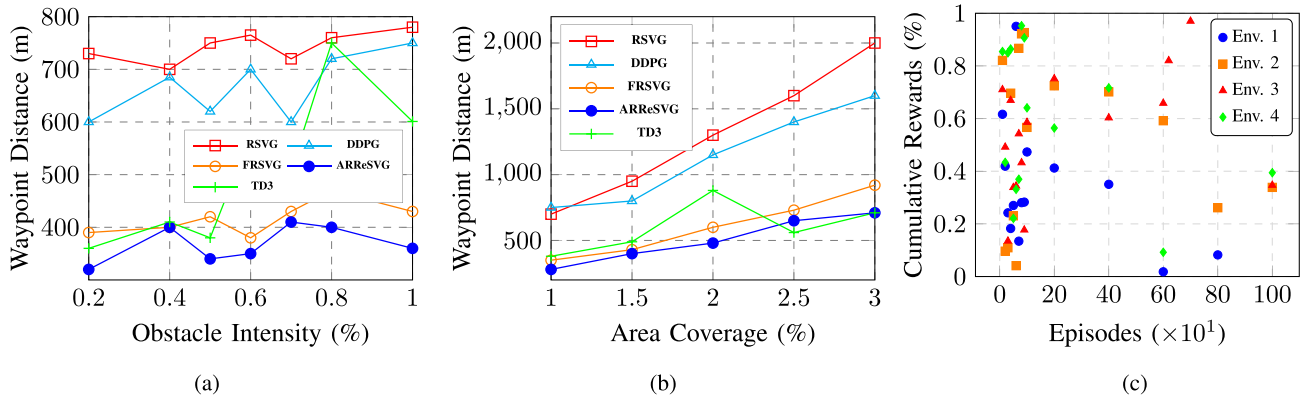

Fig. 6. Safe navigation of AAVs in the simulated partial observable space environment: (a) comparison between the waypoint distance and obstacle intensity, (b) comparison of area coverage against waypoint distance, and (c) GRU net's training impact in achieving cumulative rewards over a time period.

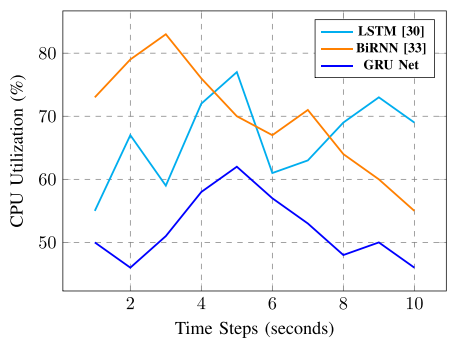

Fig. 7. Comparison of GRU with other DRL models in terms of computational throughput.

observability through recurrent structures but is computationally intensive. Similarly, FRSVG provides effective exploration but struggles in complex environments. These comparisons emphasize the novelty of ARReSVG, which integrates memory-based learning and coordination to effectively address the partial observability and multi-AAV navigation challenges.

C. Performance Analysis of GRU Net

The GRU Net's performance has been investigated to optimize the value approximation function. The combination of GRU Net and the ARReSVG framework has a significant effect on rewards in four different challenging environments (categorized based on the obstacle intensity). Fig. 6(c) compares the cumulative rewards acquired over the training episodes for four different environments. Each environment is highlighted with distinct markers. Among the environments, Environment 3 achieves the highest cumulative rewards of 97% at episode 700. Environments 1, 2, and 4 show varying rewards, fluctuating significantly across episodes. Environment 1 shows a moderate peak of 92% of rewards at episode 100. Environment 2 achieves its maximum reward of 90% at episode 99, and Environment 4 has the highest reward of 91% at episode 110.

These observations highlight the variability in performance across environments, and outperforming Environment 3 dominates in terms of cumulative rewards. The utilization of computational throughput with GRU Net against other models, such as LSTM [30] and Bi-RNN [33], is also examined in Fig. 7. Finally, integrating the GRU Net into the ARReSVG framework led to an average reduction of 30% in the computational requirements relative to the time steps. In partial observability, GRUs perform better adaptability to learn sequential dependencies among the data, help execute navigational information efficiently, and support the ARReSVG framework to provide better decision-making.

VI. Conclusion and Future Works

Experimental efforts with the ARReSVG framework and GRU Net have allowed us to formulate an efficient autonomous multi-AAV navigation system in partially observable spaces. ARReSVG leverages the multi-AAV consensus path-planning mechanism and rapid recurrent stochastic action-value policy to improve the value function. This aids the autonomy of the AAVs in the chosen large-scale environment, with a result of above 94%. In addition, the GRU Net utilized in Environment 3 contributed to approximately 97% of the cumulative rewards over the time steps, expediting trajectory optimization within the ARReSVG framework. In the future, the ARReSVG framework can incorporate online learning-based adaptability mechanisms to adjust the dynamic changes in learning rates more accurately while training in uncertain environments. Furthermore, exploration-exploitation balancing techniques must be implemented to execute efficient autonomous swarm navigation in a continuous search space.

References

[1] A. Heidari, N. J. Navimipour, M. Unal, and G. Zhang, "Machine learning applications in Internet-of-Drones: Systematic review, recent deployments, and open issues," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–45, 2023.

[2] Y. Tang et al., "Perception and navigation in autonomous systems in the era of learning: A survey," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 9604–9624, Dec. 2023.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.

[3] C. Cadena et al., "Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age," *IEEE Trans. Robot.*, vol. 32, no. 6, pp. 1309–1332, Dec. 2016.

[4] P. K. Selvam, G. Raja, V. Rajagopal, K. Dev, and S. Knorr, "Collision-free path planning for UAVs using efficient artificial potential field algorithm," in *Proc. IEEE 93rd Veh. Technol. Conf.*, 2021, pp. 1–5.

[5] S. P. Bharati, Y. Wu, Y. Sui, C. Padgett, and G. Wang, "Real-time obstacle detection and tracking for sense-and-avoid mechanism in UAVs," *IEEE Trans. Intell. Veh.*, vol. 3, no. 2, pp. 185–197, Jun. 2018.

[6] Y. Liu, Q. Wang, H. Hu, and Y. He, "A novel real-time moving target tracking and path planning system for a quadrotor UAV in unknown unstructured outdoor scenes," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 49, no. 11, pp. 2362–2372, Nov. 2019.

[7] H. Huang and A. V. Savkin, "An algorithm of reactive collision free 3-D deployment of networked unmanned aerial vehicles for surveillance and monitoring," *IEEE Trans. Ind. Informat.*, vol. 16, no. 1, pp. 132–140, Jan. 2020.

[8] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introducbon," *IEEE Trans. Neural Netw.*, vol. 9, no. 5, p. 1054, Sep. 1998.

[9] S. Qamar, S. H. Khan, M. A. Arshad, M. Qamar, and A. Khan, "Autonomous drone swarm navigabon and mulbtarget tracking with Island policy-based opbmizabon framework," *IEEE Access*, vol. 10, pp. 91073–91091, 2022.

[10] D. Liu, Q. Zong, X. Zhang, R. Zhang, L. Dou, and B. Tian, "Game of drones: Intelligent online decision making of Multi-UAV confrontation," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 2, pp. 2086–2100, Apr. 2024.

[11] M. T. Spaan, *Partially Observable Markov Decision Processes*, vol. 12. Springer, 2012, pp. 387–414.

[12] B. Yuan et al., "A UAV-Assisted search and localization strategy in non-line-of-sight scenarios," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 23841–23851, Dec. 2022.

[13] C. Wang, J. Wang, Y. Shen, and X. Zhang, "Autonomous navigation of UAVs in large-scale complex environments: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2124–2136, Mar. 2019.

[14] H. Du, S. Thudumu, R. Vasa, and K. Mouzakis, "A survey on context-aware multi-agent systems: Techniques, challenges and future directions," 2024, *arXiv:2402.01968*.

[15] G. Raja et al., "CPP-OTPO: A cognitive Multi-UAV system for adaptive path and trajectory optimization," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2024, pp. 1–6.

[16] C. Wang, J. Wang, J. Wang, and X. Zhang, "Deep-reinforcement-Learning-Based autonomous UAV navigation with sparse rewards," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6180–6190, Jul. 2020.

[17] Y.-J. Chen, D.-K. Chang, and C. Zhang, "Autonomous tracking using a swarm of UAVs: A constrained multi-agent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13702–13717, Nov. 2020.

[18] J. Moon, S. Papaioannou, C. Laoudias, P. Kolios, and S. Kim, "Deep reinforcement learning Multi-UAV trajectory control for target tracking," *IEEE Internet Things J.*, vol. 8, no. 20, pp. 15441–15455, Oct. 2021.

[19] G. Zhao, Y. Wang, T. Mu, Z. Meng, and Z. Wang, "Reinforcement learning assisted Multi-UAV task allocation and path planning for IIoT," *IEEE Internet Things J.*, vol. 11, no. 16, pp. 26766–26777, Aug. 2024.

[20] S. Ouahouah, M. Bagaa, J. Prados-Garzon, and T. Taleb, "Deep-reinforcement-Learning-Based collision avoidance in UAV environment," *IEEE Internet Things J.*, vol. 9, no. 6, pp. 4015–4030, Mar. 2022.

[21] D. Wang, T. Fan, T. Han, and J. Pan, "A two-stage reinforcement learning approach for Multi-UAV collision avoidance under imperfect sensing," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 3098–3105, Apr. 2020.

[22] S. Zhang, Y. Li, and Q. Dong, "Autonomous navigation of UAV in multi-obstacle environments based on a deep reinforcement learning approach," *Appl. Soft Comput.*, vol. 115, 2022, Art. no. 108194. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1568494621010383

[23] L. Zhang, W. Yi, H. Lin, J. Peng, and P. Gao, "An efficient reinforcement learning-based cooperative navigation algorithm for multiple UAVs in complex environments," *IEEE Trans. Ind. Informat.*, vol. 20, no. 10, pp. 12396–12406, Oct. 2024.

[24] H. X. Pham, H. M. La, D. Feil-Seifer, and L. V. Nguyen, "Autonomous UAV navigation using reinforcement learning," 2018, *arXiv:1801.05086*.

[25] G. Raja, S. Essaky, A. Ganapathisubramaniyan, and Y. Baskar, "Nexus of deep reinforcement learning and leader-follower approach for AIoT enabled aerial networks," *IEEE Trans. Ind. Informat.*, vol. 19, no. 8, pp. 9165–9172, Aug. 2023.

[26] A. Singla, S. Padakandla, and S. Bhatnagar, "Memory-based deep reinforcement learning for obstacle avoidance in UAV with limited environment knowledge," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 1, pp. 107–118, Jan. 2021.

[27] L. Petrović, J. Peršić, M. Seder, and I. Marković, "Stochastic optimization for trajectory planning with heteroscedastic gaussian processes," in *Proc. 2019 Eur. Conf. Mobile Robots*, 2019, pp. 1–6.

[28] H. Qin et al., "Autonomous exploration and mapping system using heterogeneous UAVs and UGVs in GPS-denied environments," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1339–1350, Feb. 2019.

[29] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.

[30] Y. Xue and W. Chen, "A UAV navigation approach based on deep reinforcement learning in large cluttered 3D environments," *IEEE Trans. Veh. Technol.*, vol. 72, no. 3, pp. 3001–3014, Mar. 2023.

[31] M. Lauri, D. Hsu, and J. Pajarinen, "Partially observable Markov decision processes in robotics: A survey," *IEEE Trans. Robot.*, vol. 39, no. 1, pp. 21–40, Feb. 2023.

[32] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.

[33] Z. Niu et al., "Recurrent attention unit: A new gated recurrent unit for long-term memory of important parts in sequential data," *Neurocomputing*, vol. 517, pp. 1–9, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925231222013339

[34] L. He, N. Aouf, and B. Song, "Explainable deep reinforcement learning for UAV autonomous path planning," *Aerosp. Sci. Technol.*, vol. 118, 2021, Art. no. 107052.

[35] B. Zhang, X. Lin, Y. Zhu, J. Tian, and Z. Zhu, "Enhancing Multi-UAV reconnaissance and search through double critic DDPG with belief probability maps," *IEEE Trans. Intell. Veh.*, vol. 9, no. 2, pp. 3827–3842, Feb. 2024.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on March 27,2026 at 09:03:58 UTC from IEEE Xplore. Restrictions apply.